

НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
"КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ ІМЕНІ ІГОРЯ СІКОРСЬКОГО"
НАВЧАЛЬНО-НАУКОВИХ ІНСТИТУТ АТОМНОЇ ТА ТЕПЛОВОЇ ЕНЕРГЕТИКИ
КАФЕДРА ЦИФРОВИХ ТЕХНОЛОГІЙ В ЕНЕРГЕТИЦІ

ЛАБОРАТОРНА РОБОТА №2

з дисципліни "Основи Cassandra: створення першої енергетичної системи моніторингу"

ТЕМА: "Дослідження продуктивності Apache Kafka для енергетичних IoTпотоків"

ВАРІАНТ: 6 ПІДВАРІАНТ Б

Виконав: студент групи ТР-52мп

Томін Владислав Вікторович

Перевірив: Волков О. В.

МЕТА РОБОТИ

Освоїти принципи проєктування розподілених схем даних у Apache Cassandra на прикладі енергетичних систем моніторингу. Навчитися створювати keyspace, таблиці з різними типами ключів, наповнювати їх тестовими даними, виконувати базові запити та аналізувати ефективність різних схем зберігання.

ЗАВДАННЯ РОБОТИ

Створити keyspace і таблиці для: котлів, турбін, викидів, екологічних зведень.

Порівняти варіанти ключів: block_id, fuel_type, (region, date).

Написати скрипт на Python для порівняння двох об'єктів (середня потужність).

Зробити короткий аналіз і висновки щодо придатності Cassandra для енергетичних IoT-систем.

ОПИС ЗАВДАННЯ ЗА ВАРІАНТОМ

Система: 12 енергоблоків ТЕС (вугілля/газ), Східна Україна.

Потік даних: ~кожні 10 с, ключові параметри: тиск, температура, витрата палива, потужність, оберт, викиди CO₂/NO_x/SO₂

Порівнювані схеми доступу:

1. Історія конкретного блоку → ключ block_id.
2. Еко-моніторинг за паливом → ключ fuel_type.
3. Добовий звіт за регіоном → ключ (region, date).

ХІД РОБОТИ

1. Keyspace

```
[cqlsh 6.1.0 | Cassandra 4.1.10 | CQL spec 3.4.6 | Native protocol v5]
Use HELP for help.
cqlsh> CREATE KEYSPACE IF NOT EXISTS thermal_ua
... WITH replication = {'class': 'SimpleStrategy', 'replication_factor': 1};
cqlsh> USE thermal_ua;
cqlsh:thermal_ua> CREATE TABLE IF NOT EXISTS boilers_by_block (
```

2. Створення таблиць

```
... ) WITH CLUSTERING ORDER BY (ts DESC);
cqlsh:thermal_ua> DESCRIBE TABLES;

boilers_by_block  emissions_by_block  turbines_by_block

cqlsh:thermal_ua> CREATE TABLE IF NOT EXISTS emissions_by_fuel ( fuel_type text, ts timestamp, block_id text, co2_gps double,
no_x_gps double, so2_gps double, region text, date date, PRIMARY KEY ((fuel_type), ts, block_id) ) WITH CLUSTERING
ORDER BY (ts DESC);
cqlsh:thermal_ua> CREATE TABLE IF NOT EXISTS emissions_by_region_date ( region text, date date, ts timestamp, block_id text,
fuel_type text, co2_gps double, no_x_gps double, so2_gps double, PRIMARY KEY ((region, date), ts, block_id) ) WITH C
LUSTERING ORDER BY (ts DESC);
cqlsh:thermal_ua> DESCRIBE TABLES;

boilers_by_block      emissions_by_fuel      turbines_by_block
emissions_by_block    emissions_by_region_date

cqlsh:thermal_ua> CREATE TABLE IF NOT EXISTS env_summary_by_station_date ( station_id text, date date, co2_total_t double,
no_x_total_t double, so2_total_t double, avg_fuel_flow_tph double, PRIMARY KEY ((station_id, date)) );
```

3. Внесення даних в таблицю та їх відображення

```
cqlsh:thermal_ua> INSERT INTO boilers_by_block (block_id, ts, pressure_mpa, temperature_c, fuel_flow_tph, fuel_type, region, date) VALUES ('THERMAL_DN_1', toTimestamp(now()), 24.8, 520.0, 18.5, 'coal', 'Donetsk', toDate(now()));
cqlsh:thermal_ua> INSERT INTO boilers_by_block (block_id, ts, pressure_mpa, temperature_c, fuel_flow_tph, fuel_type, region, date) VALUES ('THERMAL_KH_2', toTimestamp(now()), 25.3, 515.0, 16.9, 'gas', 'Kharkiv', toDate(now()));
cqlsh:thermal_ua> INSERT INTO turbines_by_block (block_id, ts, rpm, power_mw, load_pct) VALUES ('THERMAL_DN_1', toTimestamp(now()), 3000, 215.0, 72.0);
cqlsh:thermal_ua> INSERT INTO turbines_by_block (block_id, ts, rpm, power_mw, load_pct) VALUES ('THERMAL_KH_2', toTimestamp(now()), 3000, 228.5, 76.0);
cqlsh:thermal_ua> INSERT INTO turbines_by_block (block_id, ts, rpm, power_mw, load_pct) VALUES ('THERMAL_DN_1', toDate(now()) - 600000, 2990, 210.0, 70.0);

($ rows)
cqlsh:thermal_ua> SELECT * FROM boilers_by_block WHERE block_id='THERMAL_DN_1' LIMIT 5;

block_id | ts | date | fuel_flow_tph | fuel_type | pressure_mpa | region | temperature_c
-----
THERMAL_DN_1 | 2025-10-27 10:08:43.503800+0000 | 2025-10-27 | 18.5 | coal | 24.8 | Donetsk | 520

(1 rows)
cqlsh:thermal_ua> SELECT * FROM turbines_by_block WHERE block_id='THERMAL_KH_2' LIMIT 5;

block_id | ts | load_pct | power_mw | rpm
-----
THERMAL_KH_2 | 2025-10-27 10:31:05.182000+0000 | 77 | 232 | 3005
THERMAL_KH_2 | 2025-10-27 10:00:11.144000+0000 | 76 | 228.5 | 3000

(2 rows)
cqlsh:thermal_ua> SELECT * FROM emissions_by_block WHERE block_id='THERMAL_DN_1' LIMIT 3;

block_id | ts | co2_gps | date | fuel_type | nox_gps | region | so2_gps
-----
THERMAL_DN_1 | 2025-10-27 10:11:18.145000+0000 | 950 | 2025-10-27 | coal | 4.2 | Donetsk | 1.1

(1 rows)
cqlsh:thermal_ua> SELECT * FROM emissions_by_fuel WHERE fuel_type='coal' LIMIT 3;

fuel_type | ts | block_id | co2_gps | date | nox_gps | region | so2_gps
```

4. Python скрипт порівняння підключається до Cassandra, читає з таблиці `turbines_by_block` останні значення потужності (`power_mw`) для двох вибраних блоків (змінні `BLOCK_A`, `BLOCK_B`, за замовчуванням до 50 рядків), рахує середню потужність для кожного і друкує коротке порівняння та висновок

```
(py39-cqlsh) mrlok@DESKTOP-PRK08H0:~$ python compare_two_blocks.py
(py39-cqlsh) mrlok@DESKTOP-PRK08H0:~$ python compare_two_blocks.py
THERMAL_DN_1 - 213.3 MBт, THERMAL_KH_2 - 230.2 MBт → THERMAL_KH_2 ефективніший
Висновок: THERMAL_KH_2 має вищу середню потужність на 16.9 MBт за останні вимірювання.
(py39-cqlsh) mrlok@DESKTOP-PRK08H0:~$
```

Порівняння ключів доступу

- **Історія блока (`block_id`)** - цей ключ дозволяє миттєво отримати всі події одного блоку — читається одна партиція, швидко й без фільтрацій.
Використовуємо для оперативного моніторингу та розслідування інцидентів.
- **Еко-зріз за паливом (`fuel_type`)** - дає зріз «усі coal / усі gas» одним запитом, бо вся добірка по типу палива лежить разом.
Зручно порівнювати викиди й ефективність між паливами без повільного `ALLOW FILTERING`.
- **Денний звіт за регіоном (`region, date`)** - партиція створюється на комбінацію регіон+дата, тому денний звіт дістається адресно й дешево.
Ідеально для щоденної екозвітності та підсумкової аналітики по областях.

```

(3 rows)
/usr/bin/cqlsh.py:1008: DeprecationWarning: ResultSet indexing support will be removed in 4.0. Consider using ResultSet.one() to get a single row.
Tracing session: 29e4aa0-b325-11f0-8843-6b63875742cb

activity | source | source_elapsed | client | times
-----|-----|-----|-----|-----
10-27 11:07:44.654000 | 127.0.0.1 | 0 | 127.0.0.1 | Execute CQL3 query | 2025-
10-27 11:07:44.650000 | 127.0.0.1 | 6115 | 127.0.0.1 | Parsing SELECT block_id, ts, power_mw, load_pct FROM turbines_by_block WHERE block_id='THERMAL_OH_1' LIMIT 5; [Native-Transport-Requests-1] | 2025-
10-27 11:07:44.660000 | 127.0.0.1 | 6511 | 127.0.0.1 | Preparing statement [Native-Transport-Requests-1] | 2025-
10-27 11:07:44.660001 | 127.0.0.1 | 7048 | 127.0.0.1 | Executing single-partition query on turbines_by_block [ReadStage-3] | 2025-
10-27 11:07:44.660002 | 127.0.0.1 | 7150 | 127.0.0.1 | Acquiring stable references [ReadStage-3] | 2025-
10-27 11:07:44.661000 | 127.0.0.1 | 7276 | 127.0.0.1 | Skipped 0/0 non-slice-intersecting sstables, included 0 due to tombstones [ReadStage-3] | 2025-
10-27 11:07:44.661001 | 127.0.0.1 | 7493 | 127.0.0.1 | Merged data from memtables and 0 sstables [ReadStage-3] | 2025-
10-27 11:07:44.661002 | 127.0.0.1 | 7697 | 127.0.0.1 | Read 3 live rows and 0 tombstone cells [ReadStage-3] | 2025-
10-27 11:07:44.662211 | 127.0.0.1 | 8211 | 127.0.0.1 | Request complete | 2025-

cqlsh:thermal_uax SELECT fuel_type, ts, block_id, co2_gps, nox_gps, so2_gps FROM emissions_by_fuel WHERE fuel_type='coal' LIMIT 5;

fuel_type | ts | block_id | co2_gps | nox_gps | so2_gps
-----|-----|-----|-----|-----|-----
coal | 2025-10-27 10:11:25.752000+0000 | THERMAL_OH_1 | 950 | 4.2 | 1.1

(1 rows)

Tracing session: 44055f50-b325-11f0-8843-6b63875742cb

activity | timestamp | source | source_elapsed | client | times
-----|-----|-----|-----|-----|-----
10-27 11:08:28.485000 | 127.0.0.1 | 0 | 127.0.0.1 | Execute CQL3 query | 2025-
10-27 11:08:28.486000 | 127.0.0.1 | 472 | 127.0.0.1 | Parsing SELECT fuel_type, ts, block_id, co2_gps, nox_gps, so2_gps FROM emissions_by_fuel WHERE fuel_type='coal' LIMIT 5; [Native-Transport-Requests-1] | 2025-
10-27 11:08:28.486001 | 127.0.0.1 | 749 | 127.0.0.1 | Preparing statement [Native-Transport-Requests-1] | 2025-
10-27 11:08:28.487000 | 127.0.0.1 | 1424 | 127.0.0.1 | Executing single-partition query on emissions_by_fuel [ReadStage-3] | 2025-
10-27 11:08:28.487001 | 127.0.0.1 | 1595 | 127.0.0.1 | Acquiring stable references [ReadStage-3] | 2025-
10-27 11:08:28.487002 | 127.0.0.1 | 1594 | 127.0.0.1 | Skipped 0/0 non-slice-intersecting sstables, included 0 due to tombstones [ReadStage-3] | 2025-
10-27 11:08:28.487003 | 127.0.0.1 | 1858 | 127.0.0.1 | Merged data from memtables and 0 sstables [ReadStage-3] | 2025-
10-27 11:08:28.487004 | 127.0.0.1 | 1996 | 127.0.0.1 | Read 1 live rows and 0 tombstone cells [ReadStage-3] | 2025-
10-27 11:08:28.487349 | 127.0.0.1 | 2349 | 127.0.0.1 | Request complete | 2025-

cqlsh:thermal_uax

```

Ключ	Для чого	Чому саме
Block_id	Історія одного блоку, оперативний моніторинг	Усі події блоку лежать в одній партиції → читання без ALLOW FILTERING, мінімальна затримка; з CLUSTERING ORDER BY (ts DESC) миттєво отримуємо останні N записів.
Fuel_type	Еко-зрізи за типом палива, порівняння	Вся вибірка по одному паливу — в одній партиції - швидкі зрізи й агрегати; на схемі з block_id довелося би фільтрувати по неключовій колонці (повільно).
Region, date	Добовий звіт по регіону	Партиція = регіон + дата → адресний доступ без сканувань; дата природно «дробить» партиції, тримаючи їх малими і швидкими.

ВІДПОВІДІ НА КОНТРОЛЬНІ ПИТАННЯ

- 1. Partition vs Clustering key:**
Partition key визначає, в якій партиції зберігаються рядки (маршрутизація по кластеру).
Clustering key задає порядок всередині партиції та формує діапазонні запити (час/діапазон).
- 2. Чому денормалізація:**
Cassandra оптимізована під швидкі читання за ключем. Під кожен сценарій доступу створюють окрему таблицю, щоб уникати повільних сканів/ALLOW FILTERING.
- 3. Ключ для часових рядів:**
Партиційний ключ — ідентифікатор об'єкта (напр., block_id), clustering — час (ts) з CLUSTERING ORDER BY (ts DESC). За потреби — «бакетизація» по даті/годині, щоб партиції не розрослись.
- 4. Реплікаційний фактор (RF):**
Скільки копій кожного партиційного фрагмента зберігається в кластері. Більший

- RF → вища відмовостійкість і доступність, але більше диску/реплікаційного трафіку.
5. **Навіщо CLUSTERING ORDER BY (timestamp DESC):**
Щоб останні події отримувати без сортування і читати мінімум сторінок — критично для моніторингу.
 6. **Як зрозуміти, що партиція занадто велика:**
Повільні читання, високі затримки, nodetool tablestats показує великі Partitions/Mean/Max; потрібно розбивати (наприклад, додати date до партиційного ключа).
 7. **Переваги Cassandra для поточкових енергетичних даних:**
Горизонтальна масштабованість, висока швидкість запису, лінійне масштабування, гнучкі схеми під сценарії доступу, прості TTL і time-series патерни.

ВИСНОВКИ

Cassandra підходить для енергетичних IoT-систем завдяки швидкому запису, читанню за ключем і горизонтальній масштабованості. Денормалізація під сценарії (block_id, fuel_type, (region, date)) дозволяє отримувати потрібні зрізи без повільних фільтрацій. Для телеметрії з частими вставками й доступом “останні N подій” структура з CLUSTERING ORDER BY (ts DESC) забезпечує низьку латентність та предиктивну продуктивність.